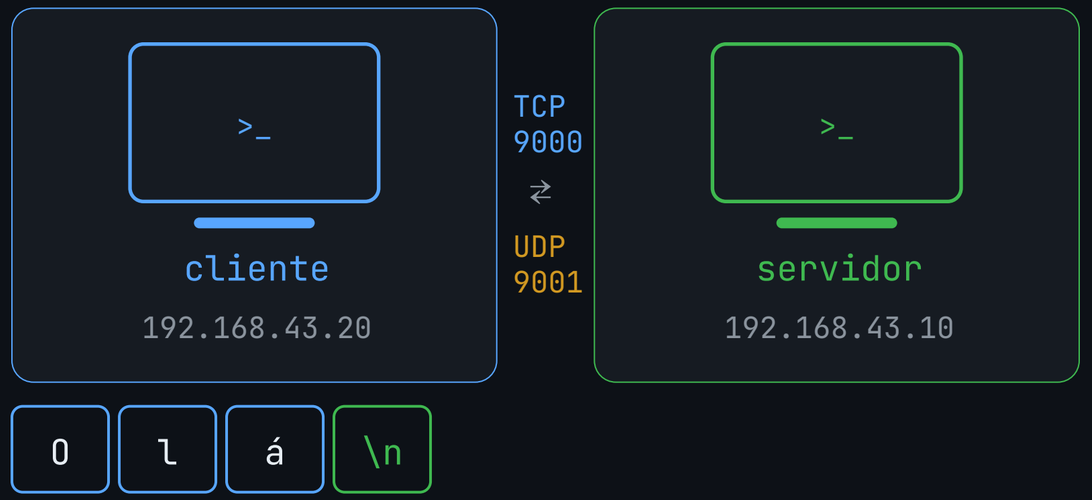
02_servidor_tcp.py  03_cliente_tcp.py  protocolo_tcp.py  04_servidor_udp.py  05_cliente_udp.py

// Redes de Computadores II

# Protocolos TCP e UDP




Aulas 06 · Redes de Computadores II

Prof. Msc. Igor Bezerra Reis  IFPI — Campus São Raimundo Nonato · TSI

// percurso

# Da comunicação confiável por fluxo aos datagramas




> Bloco 1: TCP e o protocolo da aplicação. Bloco 2: UDP e a comparação prática.

// retomada

# Revisão sobre sockets




O socket é um recurso mantido pelo sistema operacional.

O IP localiza o destino; a porta entrega ao processo.

Cliente e servidor são papéis de processos.

Na rede, tudo viaja como bytes.

// pergunta central

# Trocar bytes ainda não significa compreender mensagens

cliente

```
Cliente envia: b"OLA"
```

servidor

```
Servidor recebe: b"OLA"
```

// pergunta

Como o servidor sabe se `OLA` é uma saudação, um nome, um comando ou parte de uma mensagem maior?

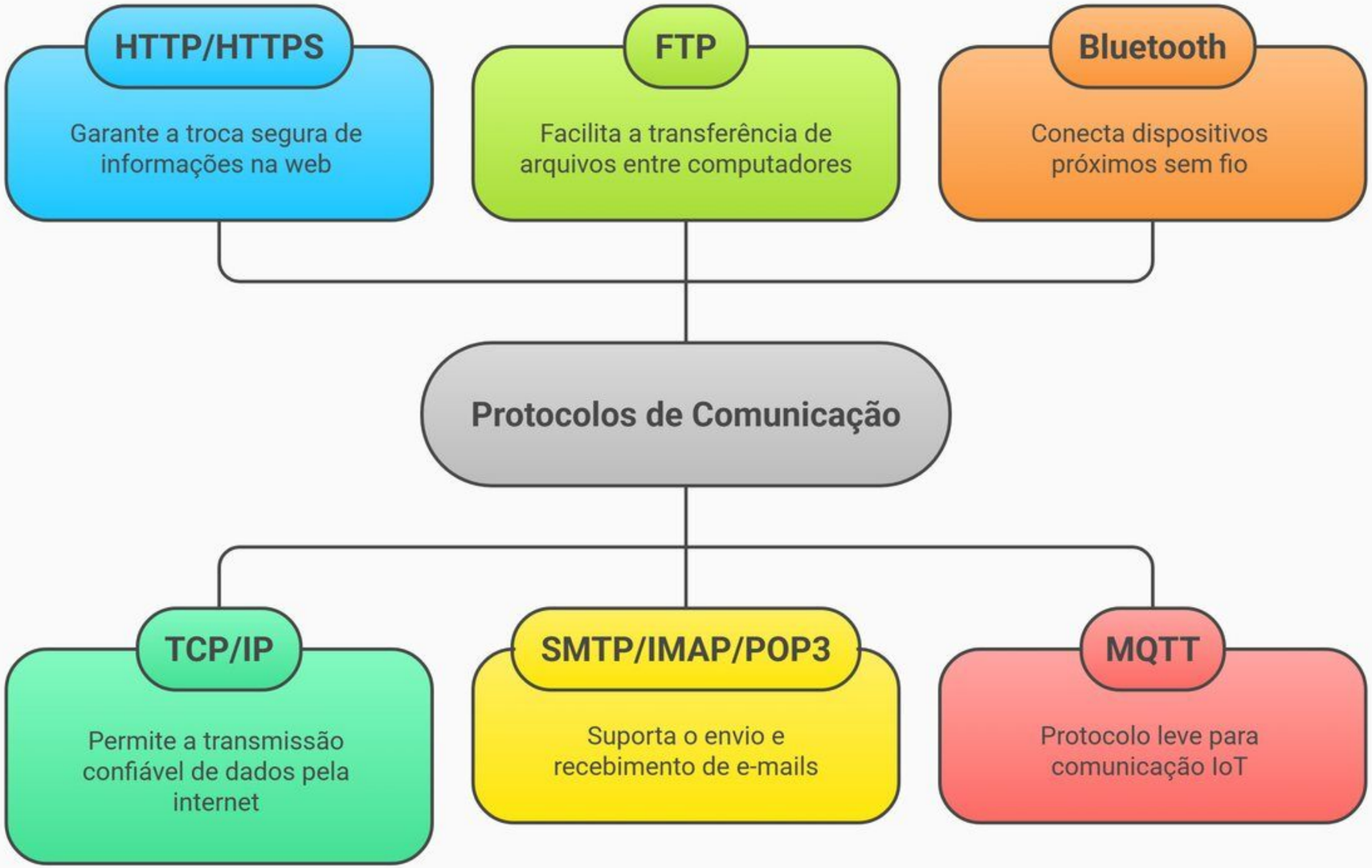
Os dois programas precisam compartilhar regras.

# Protocolo de comunicação

Um protocolo é um conjunto de regras que define como os participantes estruturam, enviam, recebem e interpretam mensagens.

- Formato · significado · ordem
- Respostas e encerramento
- Início e final da mensagem
- Mensagens inválidas

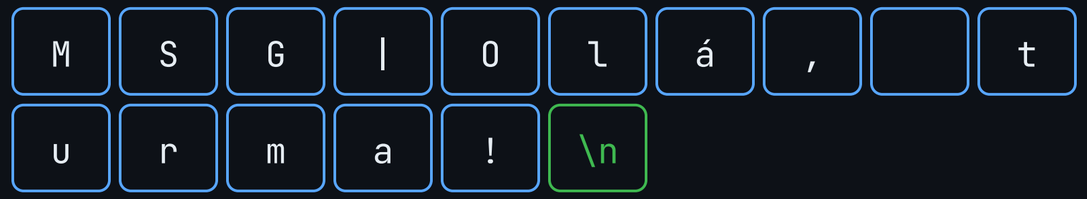
# Protocolos de comunicação: exemplos






Made with Napkin

# Perguntas que um protocolo responde




`MSG` → tipo · `|` → separa campos

`Olá, turma!` → conteúdo · `\n` → final

01 Onde a mensagem começa?

02 Onde termina?

03 Qual é o tipo?

04 Qual é o conteúdo?

05 O que o receptor deve fazer?

```
// importante
```

O TCP não define `MSG`, `HORA` ou `SAIR`. Essas regras pertencem à aplicação.

# Protocolo de aplicação, TCP e IP




Miniprotocolo → significado das mensagens.

TCP → fluxo ordenado de bytes.

IP → encaminha pacotes até o destino.

// transmission control protocol

# TCP entrega um fluxo confiável de bytes

01 Estabelece uma conexão lógica entre os processos.

02 Entrega os bytes na ordem.

03 Detecta perdas e retransmite.

04 Controla o fluxo da comunicação.

05 Avisa a aplicação quando a conexão termina ou falha.

06 Apresenta os dados como um fluxo de bytes.

# O TCP cuida do transporte; a aplicação define a conversa

| O TCP OFERECE | O TCP NÃO OFERECE |
| --- | --- |
| Entrega ordenada dos bytes | Significado das mensagens |
| Detecção e recuperação de perdas | Formato dos comandos |
| Controle da conexão | Limites entre mensagens |
| Comunicação bidirecional | Regras específicas do chat |

# O TCP cuida do transporte; a aplicação define a conversa

IP

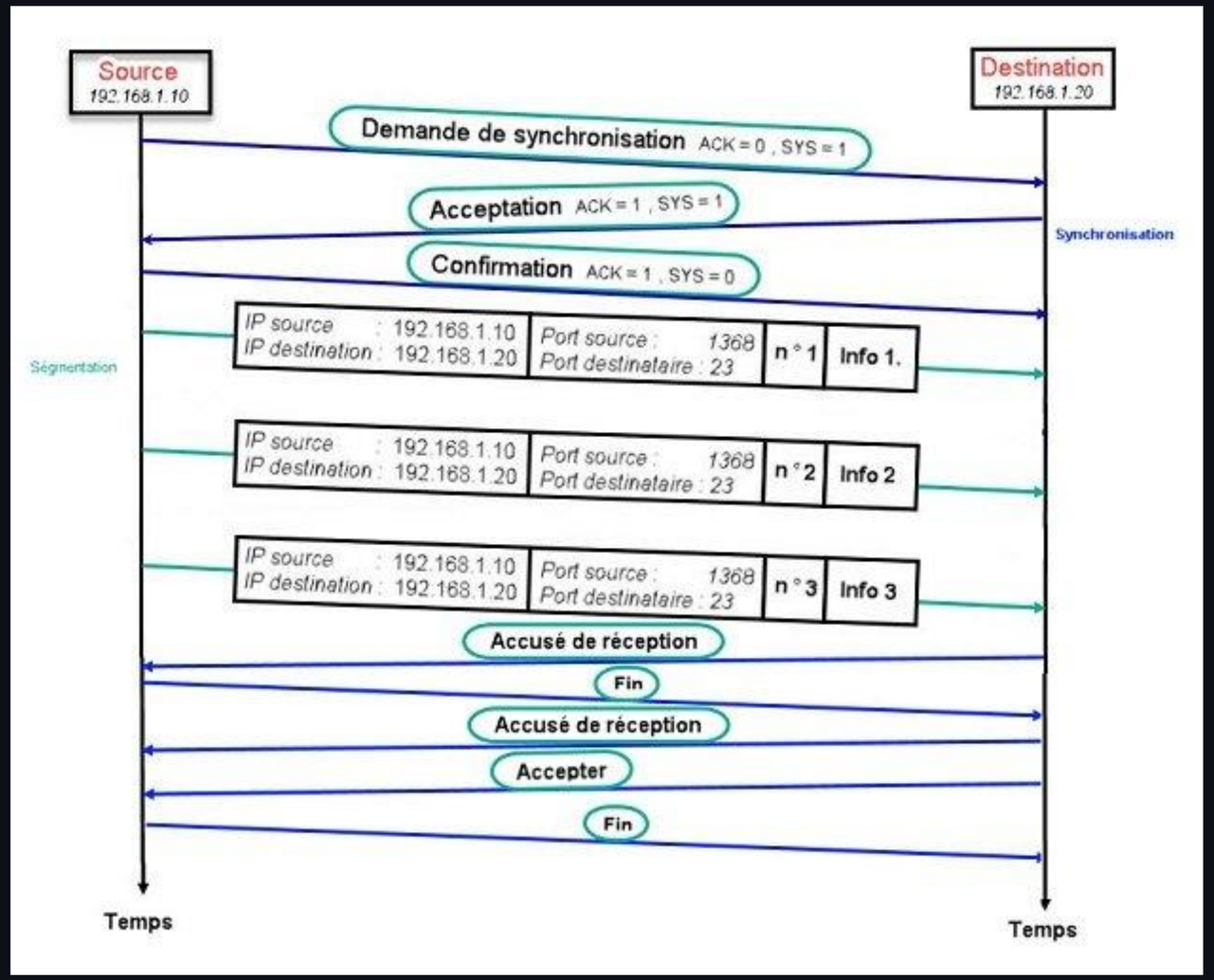
origem:  192.168.0.10
destino: 192.168.0.20

TCP

porta origem:  53000
porta destino: 9000
SEQ: 1000
ACK: 1500

Dados

OLA MUNDO



# Uma conexão é identificada pelos dois lados




> IP e porta de origem · IP e porta de destino · protocolo TCP.

// servidor tcp

# Socket TCP do servidor

02_servidor_tcp.py — criação

```
10 servidor = socket.socket(
11     socket.AF_INET,
12     socket.SOCK_STREAM
13 )
```

`AF_INET` → endereços IPv4.

`SOCK_STREAM` → fluxo de bytes.

Nesta combinação, o sistema usa TCP.

# Operações do servidor TCP

01 `socket()` cria o objeto

02 `bind()` associa IP e porta

03 `listen()` passa a aguardar conexões

04 `accept()` espera e devolve a conexão

05 `recv()` recebe bytes

06 `sendall()` envia todos os bytes

07 `close()` encerra

Socket de escuta: aguarda novas conexões.

Socket devolvido por `accept()` : conversa com aquele cliente.

# Operações do cliente TCP

01 `socket()` cria o objeto

02 `connect()` pede a conexão

03 `sendall()` envia

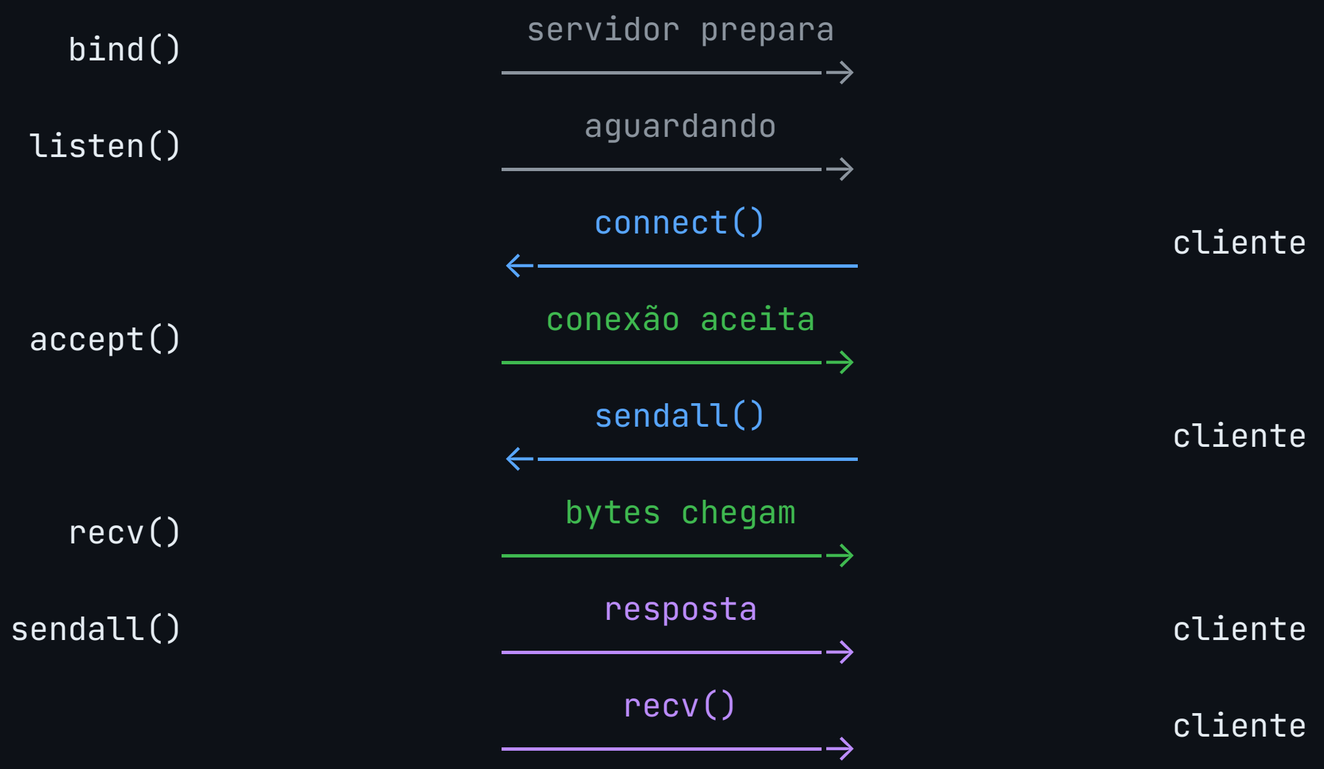
04 `recv()` recebe

05 `close()` encerra

```
// connect()
```

Solicita ao sistema operacional uma conexão com o IP e a porta do servidor.

# Estabelecimento da comunicação



# Texto e bytes

01_texto_bytes.py — encode

```
1  mensagem = "Olá"
2  dados = mensagem.encode("utf-8")
```

`sendall()` envia bytes · `recv()` devolve bytes.

01_texto_bytes.py — decode

```
4  mensagem = dados.decode("utf-8")
```

`encode()` texto → bytes · `decode()` bytes → texto.

# `recv()` retorna os bytes disponíveis

02_servidor_tcp.py — recv

```
30  dados = conexao.recv(1024)
```

`1024` é o máximo de bytes pedido naquela chamada.

```
// importante
```

Uma chamada de `recv()` não representa obrigatoriamente uma mensagem completa.

# O TCP preserva a ordem, não os limites

APLICAÇÃO ENVIA

cliente

```
sendall("OLA")
sendall("MUNDO")
```

FLUXO TCP




```
// pergunta
```

Como separar “OLA” de “MUNDO”?

# Toda mensagem termina com \n




O receptor acumula bytes até encontrar \n.

```
// importante
```

O delimitador faz parte do protocolo da aplicação, não do TCP.

# Função de envio TCP

protocolo_tcp.py — enviar_mensagem

```python
def enviar_mensagem(conexao, mensagem):
    dados = (mensagem + "\n").encode("utf-8")
    conexao.sendall(dados)
```

Acrescenta `\n` ao final.

Converte para bytes UTF-8.

`sendall()` envia todos os bytes.

# Função de recebimento TCP

protocolo_tcp.py — receber_mensagem

```python
def receber_mensagem(conexao):
    dados = b""

    while True:
        byte = conexao.recv(1)

        if not byte:
            return None

        if byte == b"\n":
            break

        dados += byte

    return dados.decode("utf-8")
```

`dados` começa vazio.

Recebe um byte por vez.

`b""` → conexão encerrada → `None`.

`\n` → fim da mensagem.

Os bytes anteriores formam o conteúdo.

# Servidor TCP básico

02_servidor_tcp.py — preparação

```python
import socket

from protocolo_tcp import enviar_mensagem
from protocolo_tcp import receber_mensagem

HOST = "0.0.0.0"
PORTA = 9000

servidor = socket.socket(
    socket.AF_INET,
    socket.SOCK_STREAM
)

servidor.bind((HOST, PORTA))
servidor.listen()

print(f"Servidor esperando em {HOST}:{PORTA}")
```

// 0.0.0.0

Aceita conexões em todas as interfaces de rede do computador.

`bind()` prende o socket à porta `9000`.

`listen()` passa a aguardar clientes.

# Aceitando um cliente

02_servidor_tcp.py — accept

```
19 conexao, endereco = servidor.accept()
20
21 print("Cliente conectado:", endereco)
```

`accept()` aguarda.

`conexao` conversa com aquele cliente.

`endereco` = IP e porta do cliente.

# O servidor devolve o que recebe

02_servidor_tcp.py — eco

```
23  while True:
24      mensagem = receber_mensagem(conexao)
25
26      if mensagem is None:
27          print("Cliente desconectou.")
28          break
29
30      print("Cliente:", mensagem)
31
32      resposta = "ECO: " + mensagem
33      enviar_mensagem(conexao, resposta)
```

Lê uma mensagem completa.

`None` → o cliente saiu.

Devolve com o prefixo `ECO:`.

O eco não precisa entender o conteúdo.

# Cliente TCP

03_cliente_tcp.py — conexão

```python
import socket

from protocolo_tcp import enviar_mensagem
from protocolo_tcp import receber_mensagem

HOST = "IP_DO_SERVIDOR"
PORTA = 9000

cliente = socket.socket(
    socket.AF_INET,
    socket.SOCK_STREAM
)

cliente.connect((HOST, PORTA))

print("Conectado ao servidor.")
```

Servidor: `0.0.0.0`.

Cliente: IP real do servidor, ex.: `192.168.43.10`.

`127.0.0.1` → o próprio computador.

# Envio e recebimento no cliente

03_cliente_tcp.py — conversa

```
18 while True:
19     mensagem = input("Mensagem: ")
20
21     enviar_mensagem(cliente, mensagem)
22
23     resposta = receber_mensagem(cliente)
24
25     print("Servidor:", resposta)
```

01 Lê o texto.

02 Envia.

03 Aguarda a resposta.

04 Mostra a resposta.

// execução

# Dois terminais, dois processos

Terminal do servidor

```
PS> py 02_servidor_tcp.py
Servidor esperando em 0.0.0.0:9000
Cliente conectado: ('192.168.43.20', 53144)
Cliente: Olá
```

Terminal do cliente

```
PS> py 03_cliente_tcp.py
Conectado ao servidor.
Mensagem: Olá
Servidor: ECO: Olá
```

> Linux: `python3 02_servidor_tcp.py` e `python3 03_cliente_tcp.py`. Servidor sempre primeiro.

// prática tcp · 15 min

# Modificação do servidor de eco

> Escolham duas modificações:

01 Converter para maiúsculas.

02 Adicionar o nome do servidor.

03 Responder a `HORA|`.

04 Encerrar com `SAIR|`.

05 Alertar se passar de 100 caracteres.

02_servidor_tcp.py — exemplo

```
32  if mensagem == "SAIR|":
33      enviar_mensagem(conexao, "Conversa encerrada")
34      break
35  elif mensagem == "HORA|":
36      enviar_mensagem(conexao, "Horário solicitado")
37  else:
38      enviar_mensagem(conexao, "ECO: " + mensagem)
```

// verificação · 5 min

# Verificação da prática TCP

TESTEM

- [ ] Mensagem normal e com acento.
- [ ] Mensagem vazia.
- [ ] Comando válido e desconhecido.
- [ ] Encerramento.
- [ ] Cliente fechado de repente.

ENTREGA

01 Cliente e servidor modificados.

02 `protocolo_tcp.py`.

03 Regras do protocolo, em poucas linhas.

04 Saídas dos dois terminais.

// 03

# User Datagram Protocol

Mesmo eco, outro transporte: datagramas em vez de fluxo.

Transporta datagramas: cada um é uma unidade enviada pela aplicação.

Sem conexão: nada de `listen()` ou `accept()`.

Não garante entrega nem ordem.

Não retransmite um datagrama perdido.

# Fluxo de bytes e datagramas

TCP · FLUXO CONTÍNUO




UDP · DATAGRAMAS




Datagrama 1: `ABCD` · Datagrama 2: `EFGH`

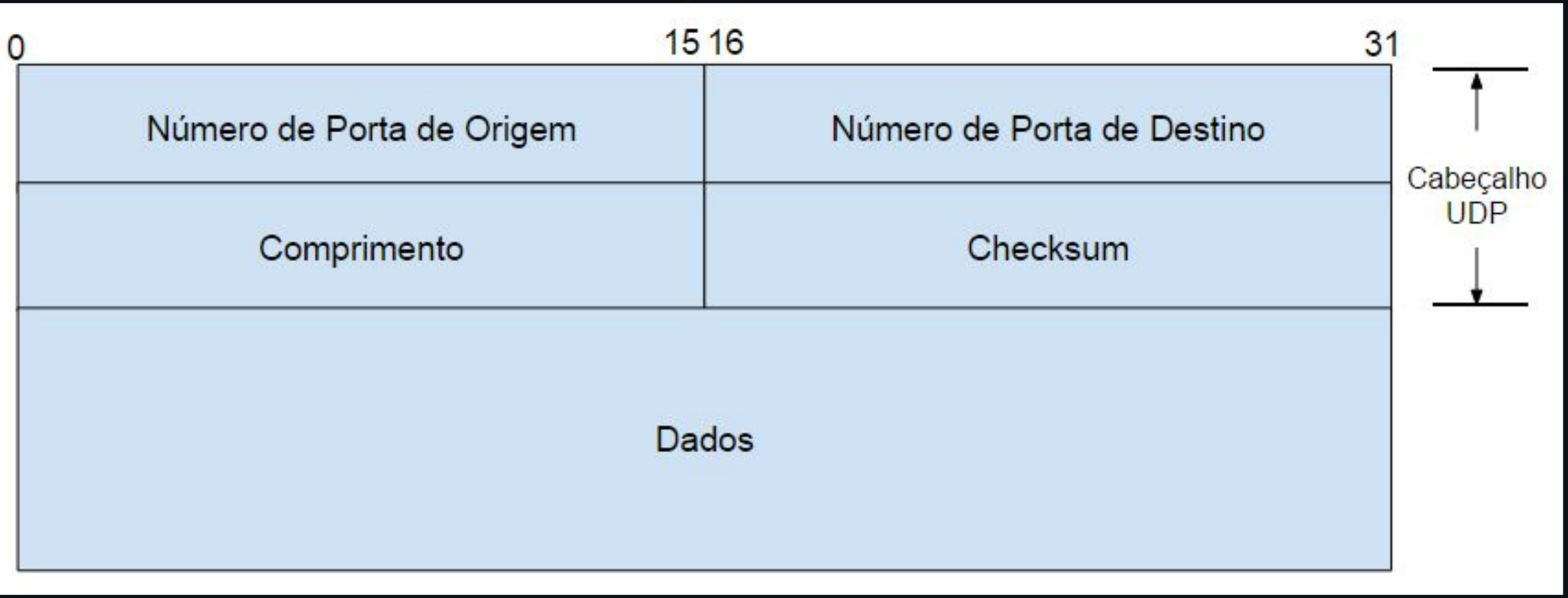
```
// limites
```

O UDP preserva os limites dos datagramas, desde que o buffer de recepção comporte o datagrama.

```
// atenção
```

Preservar limites não significa garantir entrega.

# Fluxo de bytes e datagramas



# Operações TCP e UDP

| TCP | UDP |
| --- | --- |
| `SOCK_STREAM` | `SOCK_DGRAM` |
| `listen()` | Não utiliza |
| `accept()` | Não utiliza |
| `connect()` no exemplo | Pode usar `sendto()` diretamente |
| `sendall()` | `sendto()` |
| `recv()` | `recvfrom()` |
| Fluxo de bytes | Datagramas |

// 04_servidor_udp.py

# Servidor UDP

04_servidor_udp.py — preparação

```python
import socket

HOST = "0.0.0.0"
PORTA = 9001

servidor = socket.socket(
    socket.AF_INET,
    socket.SOCK_DGRAM
)

servidor.bind((HOST, PORTA))

print(f"Servidor UDP esperando em {HOST}:{PORTA}")
```

04_servidor_udp.py — laço

```python
while True:
    dados, endereco = servidor.recvfrom(1024)

    mensagem = dados.decode("utf-8")

    print("Cliente:", endereco)
    print("Mensagem:", mensagem)

    resposta = "ECO UDP: " + mensagem

    servidor.sendto(
        resposta.encode("utf-8"),
        endereco
    )
```

> `recvfrom()` devolve os dados e o endereço de quem enviou; `sendto()` responde para esse endereço.

// 05_cliente_udp.py

# Cliente UDP

05_cliente_udp.py

```python
import socket

HOST = "IP_DO_SERVIDOR"
PORTA = 9001

cliente = socket.socket(
    socket.AF_INET,
    socket.SOCK_DGRAM
)

while True:
    mensagem = input("Mensagem: ")

    cliente.sendto(
        mensagem.encode("utf-8"),
        (HOST, PORTA)
    )

    dados, endereco = cliente.recvfrom(1024)

    resposta = dados.decode("utf-8")

    print("Servidor:", resposta)
```

Sem `connect()`, `listen()` ou `accept()`.

Cada `sendto()` informa o destino.

// cuidado

Se a resposta não chegar, `recvfrom()` espera indefinidamente.

// execução

# Execução do eco UDP

Terminal do servidor — UDP

```
PS> py 04_servidor_udp.py
Servidor UDP esperando em 0.0.0.0:9001
Cliente: ('192.168.43.20', 54001)
Mensagem: Olá
```

Terminal do cliente — UDP

```
PS> py 05_cliente_udp.py
Mensagem: Olá
Servidor: ECO UDP: Olá
```

// pergunta

O UDP funcionou neste teste. Isso prova que ele sempre entrega os dados?

// experimento controlado · 12 min

# Os mesmos testes em TCP e em UDP

01 Enviar uma palavra.

02 Enviar uma frase.

03 Enviar texto com acento.

04 Enviar mais de 100 caracteres.

05 Encerrar o servidor.

06 Tentar enviar novamente.

REGISTREM NO QUADRO

| TESTE | CHAMADA | RESULTADO | LIMITAÇÃO |
|---|---|---|---|
| … | … | … | … |

> Só `print()` e anotações. Sem arquivos de log.

# Limites de mensagem em cada protocolo

TCP

teste

```
1  parte = conexao.recv(4)
```

Devolve no máximo 4 bytes, e pode devolver menos. Não é uma mensagem completa.

UDP

teste

```
1  dados, endereco = servidor.recvfrom(1024)
```

Recebe um datagrama por vez, limitado pelo buffer.

```
// atenção
```

O resultado exato do TCP pode variar. Não registre um exemplo local como regra universal.

// comparação final

# TCP e UDP lado a lado

| CARACTERÍSTICA | TCP | UDP |
| --- | --- | --- |
| Modelo | Fluxo de bytes | Datagramas |
| Conexão | Necessária | Não existe como no TCP |
| Ordem | Preservada | Não garantida |
| Entrega | Recupera perdas com a conexão válida | Não garantida |
| Limite de mensagem | Definido pela aplicação | Preservado por datagrama |
| Chamadas | `connect accept sendall recv` | `sendto recvfrom` |
| Custo de controle | Maior | Menor |
| Exemplos | Web, APIs, arquivos, acesso remoto | DNS, áudio ao vivo, jogos, telemetria |

// projeto

# Os programas geram tráfego conhecido para o futuro IDS




IP de origem e destino · porta · protocolo.

Quantidade de bytes · frequência de mensagens.

Comandos desconhecidos · mensagens acima do limite.

// síntese e entrega

# TCP oferece fluxo; UDP oferece datagramas; a aplicação dá significado

01 Socket liga a aplicação à rede do sistema operacional.

02 TCP: fluxo ordenado de bytes.

03 O protocolo da aplicação define comandos.

04 `\n` delimita mensagens no fluxo.

05 UDP: datagramas sem as garantias do TCP.

06 Testes locais mostram observações, não garantias.

ENTREGA DA DUPLA

- [ ] TCP modificado e protocolo.
- [ ] UDP executado.
- [ ] Quadro comparativo e limitações.
- [ ] Saídas dos terminais.

> // pergunta final
> `MSG`, `HORA` e `SAIR` estão no TCP ou na aplicação?